Lecture 13 Compendium: Image Capturing & Advanced Dataset Preparation

Module 4: Practical Skills & Applied AI

Dr. Kadeghe Fue
PhD Agroecology

Compendium for Advanced Study

Contents

1 The Data-Centric Paradigm in Agriculture **2**
1.1 The GIGO Principle . . . 2

2 The Physics of Image Capture **2**
2.1 Ground Sampling Distance (GSD) in Datasets . . . 2
2.2 Lighting and BRDF . . . 2

3 Statistical Sampling Strategy **3**
3.1 Sample Size Calculation . . . 3
3.2 Stratified Random Sampling . . . 3

4 Data Annotation Standards **3**
4.1 Taxonomy & Ontology . . . 3
4.2 Annotation Formats . . . 3
4.3 Inter-Annotator Agreement (Cohen's Kappa) . . . 4

5 Advanced Data Engineering **4**
5.1 Class Imbalance Mitigation . . . 4
5.2 Data Leakage Prevention . . . 4
5.3 Augmentation Mathematics . . . 4

6 Implementation: Python Pipelines **4**
6.1 1. Augmentation Pipeline (Albumentations) . . . 5
6.2 2. Custom Dataset Loader (PyTorch) . . . 5
6.3 3. Integrity Check Script . . . 6

7 Case Study: Fall Armyworm Dataset (Tanzania) **7**
7.1 Protocol Design . . . 7
7.2 Quality Control . . . 7

8 Review Questions **7**

1 The Data-Centric Paradigm in Agriculture

In the early days of Deep Learning (2012–2018), the research focus was **Model-Centric**: designing deeper, more complex architectures (e.g., ResNet, Inception, EfficientNet) to squeeze out performance gains. Today, we exist in a **Data-Centric** era. The architecture is often a solved problem; the bottleneck is the quality, quantity, and representativeness of the data.

For a PhD student in Agroecology, this implies that your scientific contribution is less likely to be "I invented a new Neural Network layer" and more likely to be "I curated a novel, diverse, FAIR-compliant dataset that solves a specific biological problem (e.g., Cassava Brown Streak Disease detection under field conditions)."

1.1 The GIGO Principle

Garbage In, Garbage Out (GIGO) is the fundamental law of Data Science.

- **Garbage In:** Dataset with bias, wrong labels, motion blur, or lack of geographic diversity.
- **Garbage Out:** A model with high reported accuracy (99%) on the test set, but 0% utility in the real world due to overfitting or domain shift.

2 The Physics of Image Capture

To build a robust dataset, one must understand the physical process of image formation.

2.1 Ground Sampling Distance (GSD) in Datasets

When capturing images for detection (e.g., counting whiteflies), resolution matters. If the feature of interest is smaller than the GSD, it cannot be detected.

The Nyquist-Shannon Sampling Theorem applied to Vision: To resolve a feature of size L, your sampling frequency (pixel density) must be at least $2\times$ the spatial frequency of the feature. Ideally, an object should be represented by at least 10×10 pixels to be detectable by a Convolutional Neural Network (CNN).

$$GSD = \frac{S_w \cdot H}{F_r \cdot I_w} \tag{1}$$

Where:

- S_w: Sensor Width (mm)
- H: Distance to object (mm)
- F_r: Focal Length (mm)
- I_w: Image Width (pixels)

2.2 Lighting and BRDF

Vegetation interacts with light via the **Bidirectional Reflectance Distribution Function (BRDF)**. The appearance of a leaf changes based on: 1. **Illumination Angle:** Position of the sun. 2. **Viewing Angle:** Position of the camera.

PhD Implication: A dataset collected only at solar noon (nadir view) will fail to generalize to images taken at 4 PM (oblique sun angles). You must capture the **full manifold** of lighting conditions.

3 Statistical Sampling Strategy

Designing a dataset collection protocol is a statistical experiment.

3.1 Sample Size Calculation

How many images do you need? We use **Cochran's Formula** to estimate the required sample size (n_0) to estimate a proportion (e.g., prevalence of disease) with a desired level of precision.

$$n_0 = \frac{Z^2 pq}{e^2} \tag{2}$$

Where:

- Z: Z-score (1.96 for 95% confidence).
- p: Estimated proportion of the attribute (e.g., 0.1 for 10% disease prevalence).
- q: $1 - p$.
- e: Desired level of precision (e.g., 0.05 for $\pm 5\%$).

3.2 Stratified Random Sampling

To avoid selection bias, stratify the field:

- **Stratum 1 (Geography):** Highland vs. Lowland (Different soil backgrounds).
- **Stratum 2 (Phenology):** Vegetative vs. Reproductive stages.
- **Stratum 3 (Varietal):** Improved seeds vs. Local landraces.

4 Data Annotation Standards

Annotation turns raw data into information.

4.1 Taxonomy & Ontology

Before labeling, define the classes rigorously.

- **Bad Class:** "Bug" (Too vague).
- **Bad Class:** "Spodoptera frugiperda instar 3 male" (Too specific, hard for humans to label).
- **Good Class:** "Fall Armyworm" (Actionable).

4.2 Annotation Formats

1. **YOLO (Txt):** `[class_id, x_center, y_center, width, height]`. Values normalized 0-1. Best for real-time detection.
2. **COCO (JSON):** Complex hierarchy supporting Bounding Boxes, Segmentation Polygons, and Keypoints. Best for research/segmentation.
3. **Pascal VOC (XML):** Legacy format.

4.3 Inter-Annotator Agreement (Cohen's Kappa)

When multiple experts label data, they will disagree. We quantify reliability using **Cohen's Kappa** (κ).

$$\kappa = \frac{p_o - p_e}{1 - p_e} \tag{3}$$

Where:

- p_o: Relative observed agreement.
- p_e: Hypothetical probability of chance agreement.

Intersection over Union (IoU): For Bounding Boxes.

$$IoU = \frac{\text{Area of Overlap}}{\text{Area of Union}}$$

5 Advanced Data Engineering

5.1 Class Imbalance Mitigation

Agricultural datasets are naturally imbalanced (95% healthy, 5% sick).

- **Undersampling:** Randomly removing majority class examples.
- **Oversampling:** Duplicating minority class examples.
- **SMOTE (Synthetic Minority Over-sampling Technique):** Creating synthetic vectors in feature space (better than duplication).
- **Cost-Sensitive Learning:** Modifying the loss function (e.g., Weighted Cross-Entropy or Focal Loss).

5.2 Data Leakage Prevention

Spatial Leakage: Using frames from the same video in both Train and Test splits. The background is identical, so the model memorizes the background instead of the pest. **Solution:** Split by **Plot ID** or **Date**, not by random shuffle.

5.3 Augmentation Mathematics

Augmentation maps $x \to x'$ such that $Label(x) = Label(x')$.

- **Geometric:** Affine transformations (Rotation matrix R, Translation vector t).

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} t_x \\ t_y \end{bmatrix}$$

- **MixUp:** Linear interpolation between two images and their labels.

$$\tilde{x} = \lambda x_i + (1 - \lambda) x_j$$

$$\tilde{y} = \lambda y_i + (1 - \lambda) y_j$$

6 Implementation: Python Pipelines

We will implement a robust data pipeline using PyTorch and Albumentations.

6.1 1. Augmentation Pipeline (Albumentations)

This library is preferred over `torchvision` for detection because it automatically handles Bounding Box transformations.

```python
import albumentations as A
import cv2
import matplotlib.pyplot as plt

def get_train_transforms():
    return A.Compose([
        # --- Geometric ---
        A.HorizontalFlip(p=0.5),
        A.VerticalFlip(p=0.5),
        A.Rotate(limit=30, p=0.5),
        A.Perspective(scale=(0.05, 0.1), p=0.5),

        # --- Photometric ---
        A.RandomBrightnessContrast(p=0.5),
        A.HueSaturationValue(p=0.5),

        # --- Weather/Noise ---
        A.GaussNoise(var_limit=(10.0, 50.0), p=0.3),
        A.RandomShadow(p=0.3),
        A.RandomRain(p=0.1),

        # --- Regularization ---
        A.CoarseDropout(max_holes=8, max_height=32, max_width=32, p=0.3)
    ], bbox_params=A.BboxParams(format='yolo', label_fields=['class_labels']))

# Visualization Function
def visualize(image, bboxes, category_ids, category_id_to_name):
    img = image.copy()
    for bbox, category_id in zip(bboxes, category_ids):
        class_name = category_id_to_name[category_id]
        # De-normalize YOLO
        h, w, c = img.shape
        x_c, y_c, bw, bh = bbox
        x_min = int((x_c - bw/2) * w)
        y_min = int((y_c - bh/2) * h)
        x_max = int((x_c + bw/2) * w)
        y_max = int((y_c + bh/2) * h)

        cv2.rectangle(img, (x_min, y_min), (x_max, y_max), (255, 0, 0), 2)
        cv2.putText(img, class_name, (x_min, y_min-10),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.9, (255, 0, 0), 2)
    plt.figure(figsize=(10, 10))
    plt.imshow(img)
    plt.axis('off')
    plt.show()
```

Listing 1: Advanced Augmentation Pipeline

6.2 2. Custom Dataset Loader (PyTorch)

Handling data loading efficiently.

```python
import torch
from torch.utils.data import Dataset
import os
import numpy as np
from PIL import Image

```

```python
class AgricultureDataset(Dataset):
    def __init__(self, image_dir, label_dir, transforms=None):
        self.image_dir = image_dir
        self.label_dir = label_dir
        self.transforms = transforms
        self.images = [f for f in os.listdir(image_dir) if f.endswith('.jpg')]

    def __len__(self):
        return len(self.images)

    def __getitem__(self, index):
        img_path = os.path.join(self.image_dir, self.images[index])
        label_path = os.path.join(self.label_dir, self.images[index].replace('.jpg', '.txt'))

        # Load Image
        image = cv2.imread(img_path)
        image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

        # Load Labels (YOLO format: class x y w h)
        bboxes = []
        labels = []
        if os.path.exists(label_path):
            with open(label_path, 'r') as f:
                for line in f.readlines():
                    parts = list(map(float, line.strip().split()))
                    labels.append(int(parts[0]))
                    bboxes.append(parts[1:])

        # Apply Augmentations
        if self.transforms:
            transformed = self.transforms(image=image, bboxes=bboxes, class_labels=labels)
            image = transformed['image']
            bboxes = transformed['bboxes']

        # Convert to Tensor (Normalize 0-255 to 0-1)
        image = torch.from_numpy(image).permute(2, 0, 1).float() / 255.0

        # Return dictionary
        target = {}
        target['boxes'] = torch.as_tensor(bboxes, dtype=torch.float32)
        target['labels'] = torch.as_tensor(labels, dtype=torch.int64)

        return image, target

# Usage
# dataset = AgricultureDataset('data/images', 'data/labels', get_train_transforms())
```

Listing 2: PyTorch Custom Dataset Class

6.3 3. Integrity Check Script

Before training, always check for corrupt images (truncated bytes).

```python
from PIL import Image
from tqdm import tqdm

def check_dataset_integrity(directory):
    corrupt_files = []
    files = [f for f in os.listdir(directory) if f.endswith(('.jpg', '.png'))]

```

```
    print(f"Checking {len(files)} images...")
    for filename in tqdm(files):
        path = os.path.join(directory, filename)
        try:
            img = Image.open(path)
            img.verify() # Verify file integrity
        except (IOError, SyntaxError) as e:
            print(f"Corrupt file found: {filename}")
            corrupt_files.append(filename)

    return corrupt_files
```

Listing 3: Data Integrity Checker

7 Case Study: Fall Armyworm Dataset (Tanzania)

Objective: Create a dataset for FAW detection in Maize for smallholder farmers.

7.1 Protocol Design

- **Location:** 3 Regions (Kilosa, Mvomero, Morogoro Urban).
- **Device:** 50% Smartphone (Tecno/Samsung), 50% DSLR (Nikon). *Why? To simulate the deployment environment.*
- **Timing:** Weeks 3–8 (V3 to V8 stages).
- **Metadata:** Every image tagged with GPS, Date, and Scout Name.

7.2 Quality Control

After collection, 10,000 images were reviewed.

- 1,200 removed due to blur.
- 500 removed due to "No Plant Visible" (Ground shots).
- Final Dataset: 8,300 Images.
- Split: 70% Train, 20% Val, 10% Test (Spatially split by Village).

8 Review Questions

1. **Statistical Design:** You want to detect Early Blight in Tomatoes. The prevalence is 5%. Using Cochran's formula, how many plants must you scout to be 95% confident in your prevalence estimate with a 2% margin of error?
2. **Data Leakage:** Explain why randomly shuffling a video dataset of a tractor driving down a row creates data leakage. How would you split this dataset correctly?
3. **Ethics:** You pay farmers a small fee to take photos of their diseased crops. Who owns the copyright to the images? How do you credit them in the PhD thesis?

References

1. Hughes, D.P., & Salathe, M. (2015). *An open access repository of images on plant health to enable the development of mobile disease diagnostics.* ArXiv (PlantVillage).

2. Ng, A. (2021). *Data-Centric AI Competition.* DeepLearning.AI.

3. Shorten, C., & Khoshgoftaar, T. M. (2019). *A survey on image data augmentation for deep learning.* Journal of Big Data.